Kinematics

Overview

The Kinematics module (`module_kinematics.py`) provides a comprehensive set of mathematical functions for handling coordinate transformations, rotation representations, and kinematic calculations essential for marine vehicle simulation.

> **Tip**
>
> This module serves as the mathematical foundation for all spatial transformations in the simulator.

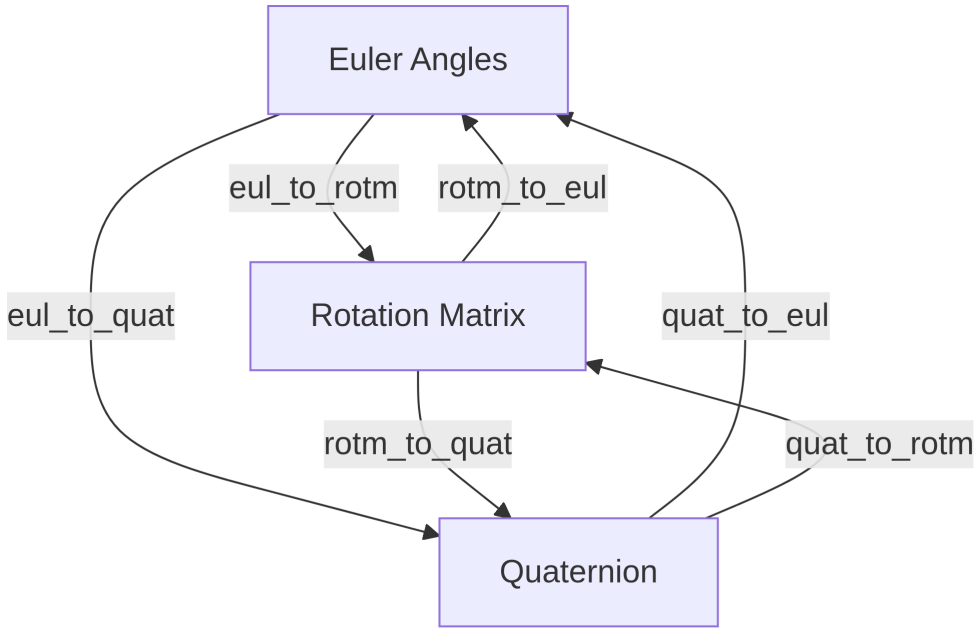
Coordinate Frames

The simulator utilizes several key coordinate frames:

1. **Earth Centered Inertial (ECI) frame {i}**: An inertial frame with its origin at the Earth's center. It does not rotate with the Earth's rotation.
2. **Earth Centered Earth Fixed (ECEF) frame {e}**: Rotates with the Earth. Its origin is at the Earth's center. The x-axis passes through the intersection of the prime meridian and the equator, the z-axis aligns with the Earth's rotation axis, and the y-axis completes the right-handed system.
3. **North-East-Down (NED) frame {n}**: A local tangent frame fixed to a point on the Earth's surface (or a reference point). The x-axis points North, the y-axis points East, and the z-axis points Down, perpendicular to the Earth's ellipsoid surface. This is the primary navigation frame.
4. **Body frame {b}**: An orthogonal frame fixed to the vehicle. The x-axis points forward, the y-axis points right (starboard), and the z-axis points down.

Function Categories

Rotation Representation Conversions



Function	Description
`eul_to_rotm(eul, order='ZYX', deg=False)`	Converts Euler angles (roll ϕ, pitch θ, yaw ψ) to a 3x3 rotation matrix that transforms vectors from the Body frame {b} to the NED frame {n}. Assumes ZYX rotation order. Angles can be in radians or degrees.
`rotm_to_eul(rotm, order='ZYX', prev_eul=None, deg=False, silent=True)`	Converts a 3x3 rotation matrix back to Euler angles (roll ϕ, pitch θ, yaw ψ). Handles potential ambiguities and gimbal lock using previous Euler angles (`prev_eul`) if provided. Can output in radians or degrees.
`eul_to_quat(eul, order='ZYX', deg=False)`	Converts Euler angles (roll ϕ, pitch θ, yaw ψ) to a unit quaternion $[q_w, q_x, q_y, q_z]$. Assumes ZYX rotation order. Angles can be in radians or degrees.
`quat_to_eul(quat, order='ZYX', deg=False, prev_quat=None, silent=True)`	Converts a unit quaternion $[q_w, q_x, q_y, q_z]$ to Euler angles (roll ϕ, pitch θ, yaw ψ). Handles potential ambiguities using `prev_quat` if provided. Can output in radians or degrees.

Function	Description
`quat_to_rotm(quat)`	Converts a unit quaternion $[q_w, q_x, q_y, q_z]$ to a 3x3 rotation matrix that transforms vectors from the Body frame {b} to the NED frame {n}.
`rotm_to_quat(rotm)`	Converts a 3x3 rotation matrix (Body to NED) to a unit quaternion $[q_w, q_x, q_y, q_z]$.

Quaternion Operations

Function	Description
`quat_multiply(q1, q2)`	Multiplies two quaternions $\mathbf{q}_1 \otimes \mathbf{q}_2$. Order matters: represents rotation $\mathbf{q}_2$ followed by rotation $\mathbf{q}_1$.
`quat_conjugate(quat)`	Computes the conjugate of a quaternion $\mathbf{q}^* = [q_w, -q_x, -q_y, -q_z]^T$.
`rotate_vec_by_quat(vec_a, q_a_b)`	Rotates a 3D vector `vec_a` using the quaternion rotation `q_a_b` (representing rotation from frame A to frame B) to get the vector in frame B. Computes $\mathbf{v}'_b = \mathbf{q}_{a\to b} \otimes \mathbf{v}'_a \otimes \mathbf{q}^*_{a\to b}$.

Rate Calculations

Function	Description
`eul_rate_matrix(eul, order='ZYX', deg=False)`	Computes the 3x3 transformation matrix $\mathbf{T}(\eta)$ that relates body-frame angular velocity ω^b to Euler angle rates $\dot{\eta}$ via $\dot{\eta} = \mathbf{T}(\eta)\omega^b$. Assumes ZYX order.
`quat_rate_matrix(quat)`	Computes the 4x3 transformation matrix $\mathbf{E}(\mathbf{q})$ that relates body-frame angular velocity ω^b to quaternion rates $\dot{\mathbf{q}}$ via $\dot{\mathbf{q}} = \frac{1}{2}\mathbf{E}(\mathbf{q})\omega^b$.
`eul_rate(eul, w, order='ZYX')`	Calculates Euler angle rates $[\dot{\phi}, \dot{\theta}, \dot{\psi}]$ given current Euler angles `eul` and body-frame angular velocity `w` $= [p, q, r]^T$. Uses `eul_rate_matrix`.
`quat_rate(quat, w)`	Calculates quaternion rates $[\dot{q}_w, \dot{q}_x, \dot{q}_y, \dot{q}_z]$ given the current quaternion `quat` and body-frame angular velocity `w` $= [p, q, r]^T$. Uses `quat_rate_matrix`.
`deul_dquat(quat)`	Computes the 3x4 Jacobian matrix $\frac{\partial \eta}{\partial \mathbf{q}}$, representing the partial derivatives of Euler angles with respect to quaternion components.

Function	Description
`dquat_deul(quat)`	Computes the 4x3 Jacobian matrix $\frac{\partial \mathbf{q}}{\partial \eta}$, representing the partial derivatives of quaternion components with respect to Euler angles. Requires converting `quat` to `eul` internally first.

Navigation Functions

Function	Description
`ssa(ang, deg=False)`	Converts an angle (in radians or degrees) to its smallest signed angle representation within the range $(-\pi, \pi]$ radians or $(-180, 180]$ degrees.
`clip(value, threshold)`	Limits the input `value` to the range `[-threshold, threshold]`.
`ned_to_llh(ned, llh0)`	Converts a position vector `ned` = [North, East, Down] relative to a reference point `llh0` = [lat0, lon0, h0] into absolute geodetic coordinates `llh` = [latitude, longitude, height]. Uses WGS84 ellipsoid model.
`llh_to_ned(llh, llh0)`	Converts an absolute geodetic position `llh` = [lat, lon, h] into a local NED position vector `ned` = [North, East, Down] relative to a reference point `llh0` = [lat0, lon0, h0]. Uses WGS84 ellipsoid model.
`generate_waypoints()`	Generates a predefined sequence of waypoints as LLH coordinates for a rectangular survey pattern relative to a specific datum location (IITM lake). Returns a numpy array of [lat, lon, height] waypoints.
`rotm_ned_to_ecef(llh)`	Computes the 3x3 rotation matrix $\mathbf{R}_n^e$ that transforms vectors from the local NED frame (defined at `llh`) to the ECEF frame.

Utility Functions

Function	Description
`Smat(vec)`	Creates the 3x3 skew-symmetric matrix $\mathbf{S}(\mathbf{v})$ corresponding to the input 3D vector `vec`. Used for representing cross products as matrix multiplications ($\mathbf{a} \times \mathbf{b} = \mathbf{S}(\mathbf{a})\mathbf{b}$).

Usage Examples

Coordinate Transformations

```python
import numpy as np
from mav_simulator.module_kinematics import eul_to_rotm, llh_to_ned

# Convert Euler angles to rotation matrix
euler_angles = np.array([0.1, 0.2, 0.3])  # [roll, pitch, yaw] in radians
R = eul_to_rotm(euler_angles)  # Rotation matrix

# Convert latitude, longitude, height to NED coordinates
reference_point = np.array([60.0, 10.0, 0.0])  # [lat, lon, height]
target_point = np.array([60.001, 10.001, 10.0])  # [lat, lon, height]
ned_coords = llh_to_ned(target_point, reference_point)
```

Rotation Representations

```python
from mav_simulator.module_kinematics import eul_to_quat, quat_to_rotm

# Convert from Euler angles to quaternion
euler_angles = np.array([0.1, 0.2, 0.3])  # [roll, pitch, yaw] in radians
quaternion = eul_to_quat(euler_angles)

# Convert from quaternion to rotation matrix
R = quat_to_rotm(quaternion)
```

Best Practices

- Use the `ssa()` function to normalize angles when working with Euler angles

- Be consistent with the rotation order convention throughout your code (the default is 'ZYX')
- When transforming between frames, always keep track of the reference frames involved